

Flight Reservation - AWS DevOps Project

Complete Step-by-Step AWS Deployment Runbook

Terraform -> AWS Infrastructure -> EKS -> ECR -> RDS -> S3 -> Jenkins/SonarQube -> GitOps/Argo CD -> Prometheus/Grafana

AWS Region	ap-south-1 (Mumbai)
Project	flight-reservation
Environment	dev
Repository	https://github.com/AnuragPatil-cloud/flight-reservation-app-AWS.git
Compute	2 EC2 C7i-flex.large VMs + 2 EKS C7i-flex.large nodes
Database	Private Amazon RDS MariaDB
Container registry	Amazon ECR
Monitoring	Prometheus + Grafana + Alertmanager + node-exporter + kube-state-metrics

Core delivery line: GitHub -> Jenkins -> SonarQube -> Docker -> ECR -> GitOps -> Argo CD -> EKS -> RDS/S3

Security note: This runbook intentionally uses placeholders for passwords, tokens, PEM paths, public IPs, and endpoints. Never commit secrets to GitHub.

1. Project Architecture and Flow

This runbook follows the AWS implementation used for the Flight Reservation application. The target architecture uses two EC2 VMs, an EKS cluster with two worker nodes, private RDS MariaDB, Amazon ECR, S3, SNS/CloudWatch, Argo CD, Prometheus and Grafana.

Component	Role
VM1	Jenkins, Docker, SonarQube, Git, Maven, AWS CLI
VM2	AWS CLI, kubectl, Helm, Argo CD CLI, EKS administration, monitoring administration
EKS	Runs frontend, reservation backend, and check-in backend
RDS	Production relational data: flightdb + checkin_db
ECR	Stores reservation, check-in, and frontend container images
S3	Persistent application files/objects
Argo CD	Synchronizes GitOps manifests from GitHub into EKS
Prometheus	Collects Kubernetes/application metrics
Grafana	Visualizes Prometheus metrics
CloudWatch/SNS	AWS infrastructure alarms and email notifications

Application workloads kept in the AWS GitOps version: flight-frontend, flight-reservation-app, flight-checkin-app. The AWS version does not deploy MariaDB as Kubernetes Deployment/PVC; RDS is the production database.

2. Quick Checklist / Order of Work

- Prepare Terraform host and AWS CLI.
- Install Terraform and validate the modular structure.
- Create secure terraform.tfvars and review the Terraform plan.
- Apply the reviewed Terraform plan.
- Verify VPC, EC2, EKS, RDS, S3, ECR, SNS and CloudWatch.
- Configure VM1: Jenkins + Docker + SonarQube.
- Configure VM2: kubectl + Helm + Argo CD CLI.
- Install Argo CD in EKS and create the flight-reservation Application.
- Verify RDS and create checkin_db.
- Verify S3 and ECR.
- Prepare GitOps ConfigMap/Secret/deployments/services.
- Run Jenkins to build, scan/analyze, push images, and update GitOps.
- Let Argo CD synchronize the application into EKS.
- Install kube-prometheus-stack and verify Prometheus/Grafana.
- Validate the application and verify data reaches RDS.
- Configure/verify CloudWatch and SNS alerts.
- Capture screenshots/evidence for GitHub.

3. Phase 1 - Prepare the Terraform Host

3.1 Clone the repository

```
cd ~
git clone https://github.com/AnuragPatil-cloud/flight-reservation-app-AWS.git
cd ~/flight-reservation-app-AWS/Terraform
```

3.2 Verify Terraform structure

```
find . -maxdepth 2 -type f | sort
find modules -maxdepth 1 -type d | sort
```

Expected module directories include: vpc, security-groups, iam, ec2-jenkins, ec2-monitoring, eks, rds, s3, ecr, sns and cloudwatch.

3.3 Install AWS CLI v2 when needed

```
apt-get update
apt-get install -y unzip curl
cd /tmp
curl -fsSL https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip -o awscliv2.zip
unzip -q -o awscliv2.zip
/tmp/aws/install --update
aws --version
```

3.4 Verify AWS access

```
aws sts get-caller-identity
aws configure
```

Region: ap-south-1

4. Phase 2 - Install Terraform

```
apt-get update
apt-get install -y gnupg wget lsb-release
rm -f /usr/share/keyrings/hashicorp-archive-keyring.gpg
rm -f /etc/apt/sources.list.d/hashicorp.list
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | tee /usr/share/keyrings/hashicorp-
archive-keyring.gpg > /dev/null
ARCH=$(dpkg --print-architecture)
CODENAME=$( . /etc/os-release && echo "${UBUNTU_CODENAME:-$(lsb_release -cs)}" )
cat <<EOF > /etc/apt/sources.list.d/hashicorp.list
deb [arch=$ARCH signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.
hashicorp.com $CODENAME main
EOF
apt-get update
apt-get install -y terraform
terraform version
```

4.1 Initialize and validate

```
cd ~/flight-reservation-app-AWS/Terraform
terraform init
terraform fmt -recursive
terraform validate
```

Do not apply before reviewing a new plan.

5. Phase 3 - Configure terraform.tfvars

Use the project values below as the baseline. Replace every placeholder with your own environment value. The RDS password, Grafana password and alert email are sensitive.

```
aws_region = "ap-south-1"
project_name = "flight-reservation"
environment = "dev"
vpc_cidr = "10.0.0.0/16"
availability_zones = ["ap-south-1a", "ap-south-1b"]
public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
private_subnet_cidrs = ["10.0.11.0/24", "10.0.12.0/24"]
database_subnet_cidrs = ["10.0.21.0/24", "10.0.22.0/24"]
instance_type = "c7i-flex.large"
jenkins_volume_size = 40
monitoring_volume_size = 40
key_name = "YOUR_EXISTING_AWS_KEYPAIR"
admin_cidr = "YOUR_PUBLIC_IP/32"
eks_node_instance_type = "c7i-flex.large"
eks_desired_nodes = 2
eks_min_nodes = 2
eks_max_nodes = 2
rds_instance_class = "db.t3.micro"
rds_engine = "mariadb"
rds_engine_version = "10.11"
rds_database_name = "flightdb"
rds_username = "flightadmin"
rds_password = "YOUR_STRONG_RDS_PASSWORD"
rds_multi_az = false
rds_backup_retention_days = 1
s3_bucket_prefix = "flight-reservation-app-dev-files"
s3_force_destroy = false
ecr_repositories = [
    "flight-reservation-dev-reservation",
    "flight-reservation-dev-checkin",
    "flight-reservation-dev-frontend"
]
sns_topic_name = "flight-reservation-dev-alerts"
alert_email = "YOUR_ALERT_EMAIL"
cloudwatch_period = 300
ec2_cpu_threshold = 80
rds_cpu_threshold = 80
rds_free_storage_threshold_bytes = 5368709120
grafana_admin_password = "YOUR_STRONG_GRAFANA_PASSWORD"
```

5.1 Find public IP for admin_cidr

```
curl -4 ifconfig.me
```

5.2 Find the EC2 key-pair name

```
aws ec2 describe-key-pairs --region ap-south-1 --query 'KeyPairs[].KeyName' --output table
```

5.3 Protect sensitive Terraform files

```
cat <<'EOF' > .gitignore
terraform.tfvars
*.tfstate
*.tfstate.*
.terraform/
crash.log
crash.*.log
EOF
```

Never commit: terraform.tfvars, .tfstate, AWS keys, database passwords, Grafana passwords or GitHub tokens.

6. Phase 4 - Terraform Plan and Apply

```
cd ~/flight-reservation-app-AWS/Terraform
terraform fmt -recursive
terraform validate
terraform plan
terraform plan -out=flight-reservation-dev.tfplan
terraform show -no-color flight-reservation-dev.tfplan | tail -40
```

Review the plan for unexpected destroys/replacements. Only after the plan is understood, apply exactly the reviewed plan:

```
terraform apply "flight-reservation-dev.tfplan"
terraform output
```

If configuration changes are required after a failed apply, fix the code, generate a fresh plan, inspect it again, and then apply the fresh plan.

7. Phase 5 - Verify AWS Infrastructure

7.1 EC2

```
aws ec2 describe-instances \
  --region ap-south-1 \
  --filters "Name=tag:Project,Values=flight-reservation" \
  --query 'Reservations[].Instances[].{Name:Tags[?Key==`Name`][0].Value, InstanceId:InstanceId, State:State.Name, PrivateIP:PrivateIpAddress, PublicIP:PublicIpAddress, Type:InstanceType}' \
  --output table
```

7.2 EKS

```
aws eks describe-cluster --name flight-reservation-dev-eks --region ap-south-1 --query 'cluster.{Name:name, Status:status, Version:version, Endpoint:endpoint}'
aws eks list-clusters --region ap-south-1
```

7.3 RDS

```
aws rds describe-db-instances \
  --region ap-south-1 \
  --db-instance-identifier flight-reservation-dev-rds \
  --query 'DBInstances[0].{Status:DBInstanceStatus, Endpoint:Endpoint.Address, Port:Endpoint.Port, Engine:Engine, Public:PubliclyAccessible}' \
  --output table
```

7.4 S3, ECR, SNS, CloudWatch

```
aws s3api list-buckets --query 'Buckets[].Name' --output table
aws ecr describe-repositories --region ap-south-1 --query 'repositories[].repositoryName' --output table
aws sns list-topics --region ap-south-1
aws cloudwatch describe-alarms --region ap-south-1 --query 'MetricAlarms[].AlarmName' --output table
```

8. Phase 6 - VM1: Jenkins + Docker + SonarQube

VM1 is the CI machine. It runs Jenkins, Docker, Git, Maven, AWS CLI and SonarQube.

8.1 Verify core tools

```
java -version
docker --version
git --version
mvn -version
aws --version
sudo systemctl status jenkins --no-pager
docker ps
```

8.2 Jenkins credentials used by this project

Credential ID	Use
github	Username + GitHub PAT/password credential used for GitOps push
sonarqube-token	Secret text containing the SonarQube token

In Jenkins Global Tool/System configuration, the SonarQube server installation name is **Sonarqube**. The server URL format must be exactly **http://HOST:9000**. Do not enter **http://http://...**

8.3 Jenkins plugin

Install the Jenkins SonarQube Scanner integration so the Pipeline step withSonarQubeEnv is available: Manage Jenkins -> Plugins -> Available plugins -> SonarQube Scanner. Then configure the SonarQube server under Manage Jenkins -> System.

8.4 Jenkins pipeline flow used in this project

1. Verify AWS
2. Backend Build - Reservation
3. Backend Build - Check-In
4. Frontend Build
5. SonarQube Analysis
6. Docker Build
7. ECR Login
8. Tag Images
9. Push Images to ECR
10. Update GitOps
11. Docker Cleanup

```
Reservation build: mvn -B clean package -DskipTests
Check-In build:   mvn -B clean package -DskipTests
Frontend build:   npm ci + npm run build
Frontend API URL: VITE_API_URL=/api
```

The implemented Jenkins pipeline intentionally skips backend unit tests and frontend linting in the deployment path. H2 test configuration exists for isolated tests, but the current CI build uses -DskipTests.

9. Backend Build Configuration - H2 Test Isolation

Both Spring Boot backends use H2 only for test-time database isolation. Production/runtime still uses RDS MySQL/MariaDB.

9.1 Add H2 once per backend POM

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>
```

Important: do not declare the H2 dependency twice. Duplicate declarations produce a Maven warning such as "must be unique: com.h2database:h2".

9.2 Reservation test properties

```
mkdir -p ~/flight-reservation-app-AWS/FlightReservationApplication/src/test/resources
cat > ~/flight-reservation-app-AWS/FlightReservationApplication/src/test/resources/application.
properties <<'EOF'
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
EOF
```

9.3 Check-In test properties

```
mkdir -p ~/flight-reservation-app-AWS/FlightCheckInApplication/src/test/resources
cat > ~/flight-reservation-app-AWS/FlightCheckInApplication/src/test/resources/application.properties
<<'EOF'
spring.datasource.url=jdbc:h2:mem:checkintestdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
EOF
```

9.4 Correct file layout

```
FlightReservationApplication/
  src/main/resources/application.properties      # runtime/RDS
  src/test/resources/application.properties      # tests/H2

FlightCheckInApplication/
  src/main/resources/application.properties      # runtime/RDS
  src/test/resources/application.properties      # tests/H2
```

Do not keep a stray project-root application.properties containing an RDS URL. In the implemented Reservation troubleshooting, a root-level file caused H2 tests to receive the RDS hostname with the H2 driver.

10. Phase 7 - VM2: kubectl + Helm + Argo CD

10.1 Verify tools

```
aws --version
kubectl version --client
helm version
argocd version --client
```

10.2 Configure EKS kubeconfig

```
aws eks update-kubeconfig --name flight-reservation-dev-eks --region ap-south-1
kubectl get nodes -o wide
```

Expected: two Ready EKS worker nodes using c7i-flex.large.

11. Phase 8 - Install Argo CD

```
kubectl create namespace argocd
kubectl apply -n argocd \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl get pods -n argocd
kubectl get svc argocd-server -n argocd
```

11.1 Get initial password when applicable

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d; echo
```

11.2 Argo CD application

```
argocd app create flight-reservation \
  --repo https://github.com/AnuragPatil-cloud/flight-reservation-app-AWS.git \
  --revision main \
  --path gitops \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace flight-reservation \
  --sync-policy automated \
  --auto-prune \
  --self-heal
```

```
argocd app get flight-reservation
argocd app sync flight-reservation
```

If the ApplicationSet controller enters CrashLoopBackOff because CRDs are missing, the implementation fix used was to install the Argo CD v3.5.3 CRDs with server-side apply, then restart the affected controller.

```
kubectl apply --server-side --force-conflicts -k "https://github.com/argoproj/argo-cd/manifests/
crds?ref=v3.5.3"
kubectl rollout restart deployment argocd-applicationset-controller -n argocd
```

12. Phase 9 - RDS and Application Databases

Database	Backend
flightdb	Reservation backend
checkin_db	Check-In backend

RDS is private and listens on port 3306. The Terraform-created database is flightdb. Create checkin_db from a host that can reach the private endpoint, such as VM1.

12.1 Create checkin_db

```
mysql -h <RDS_ENDPOINT> -P 3306 -u flightadmin -p
```

```
CREATE DATABASE IF NOT EXISTS checkin_db;  
exit;
```

12.2 Verify database access

```
mysql -h <RDS_ENDPOINT> -u flightadmin -p flightdb -e "SHOW TABLES;"  
mysql -h <RDS_ENDPOINT> -u flightadmin -p checkin_db -e "SHOW TABLES;"
```

Do not paste a real RDS password into GitHub, Jenkinsfiles, GitOps YAML or this runbook.

13. Phase 10 - S3 File Storage

S3 is used for important application files. Keep block public access enabled, server-side encryption enabled, versioning enabled and force_destroy=false for the normal project configuration.

```
aws s3api list-buckets --query 'Buckets[].Name' --output table
```

14. Phase 11 - ECR Container Images

```
flight-reservation-dev-reservation  
flight-reservation-dev-checkin  
flight-reservation-dev-frontend
```

14.1 Manual login pattern

```
aws ecr get-login-password --region ap-south-1 | \  
docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com
```

14.2 Image naming pattern

```
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/flight-reservation-dev-reservation:<BUILD_NUMBER>  
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/flight-reservation-dev-checkin:<BUILD_NUMBER>  
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/flight-reservation-dev-frontend:<BUILD_NUMBER>
```

15. Phase 12 - GitOps on EKS

The GitOps directory contains namespace, ConfigMap, Secret template, three Deployments, three Services and kustomization. In the AWS version, do not deploy MariaDB manifests.

15.1 Expected GitOps files

```
gitops/
  namespace.yaml
  configmap.yaml
  secret.example.yaml
  reservation-deployment.yaml
  reservation-service.yaml
  checkin-deployment.yaml
  checkin-service.yaml
  frontend-deployment.yaml
  frontend-service.yaml
  kustomization.yaml
```

15.2 Kustomization

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: flight-reservation
resources:
  - namespace.yaml
  - configmap.yaml
  - reservation-deployment.yaml
  - reservation-service.yaml
  - checkin-deployment.yaml
  - checkin-service.yaml
  - frontend-deployment.yaml
  - frontend-service.yaml
```

15.3 ConfigMap - use runtime values

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: flight-reservation-config
  namespace: flight-reservation
data:
  SPRING_DATASOURCE_URL_RESERVATION: "jdbc:mysql://<RDS_ENDPOINT>:3306/
flightdb?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true"
  SPRING_DATASOURCE_URL_CHECKIN: "jdbc:mysql://<RDS_ENDPOINT>:3306/
checkin_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true"
  BOOKING_SERVICE_URL: "http://flight-reservation-service:8080/api/bookings"
  FRONTEND_URL: "http://<FRONTEND_LOADBALANCER_HOSTNAME>"
```

The frontend LoadBalancer hostname is not known until the frontend Service is created. Get it with: `kubectl get svc -n flight-reservation`. Update the ConfigMap in Git, commit, push, then let Argo CD synchronize.

15.4 Secret template

```
apiVersion: v1
kind: Secret
metadata:
  name: flight-reservation-secrets
  namespace: flight-reservation
type: Opaque
stringData:
  DB_USERNAME: "<DB_USERNAME>"
  DB_PASSWORD: "<DB_PASSWORD>"
```

Keep `secret.example.yaml` as a template only. The real Secret used in this project is created directly on VM2 and is not committed to Git.

15.5 Create the real Secret on VM2

```
kubectl create secret generic flight-reservation-secrets \  
  --from-literal=DB_USERNAME=flightadmin \  
  --from-literal=DB_PASSWORD='<YOUR_REAL_RDS_PASSWORD>' \  
  -n flight-reservation
```

16. Backend Runtime application.properties

16.1 Reservation backend - src/main/resources/application.properties

```
spring.application.name=FlightReservationApplication
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.datasource.auto-commit=true
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
server.address=0.0.0.0
frontend.url=${FRONTEND_URL}
```

16.2 Check-In backend - src/main/resources/application.properties

```
spring.application.name=FlightCheckInApplication
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
booking.service.url=${BOOKING_SERVICE_URL:http://flight-reservation-service:8080/api/bookings}
server.port=8081
frontend.url=${FRONTEND_URL}
```

Runtime rule: database credentials come from Kubernetes Secret, URLs and service settings come from Kubernetes ConfigMap/environment variables. H2 is for tests only.

17. Jenkins GitOps Release

After code/config changes are pushed to main, Jenkins checks out the repository, builds the two Spring Boot applications, builds the frontend, runs SonarQube analysis, builds Docker images, pushes images to ECR, updates the three GitOps deployment image references with the Jenkins build number, and pushes the GitOps commit back to GitHub.

17.1 Verify GitHub before build

```
cd ~/flight-reservation-app-AWS
git pull origin main
git status
```

17.2 Jenkins update-GitOps logic

```
AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
ECR_REGISTRY="${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"
RESERVATION_IMAGE="${ECR_REGISTRY}/${RESERVATION_REPO}:${IMAGE_TAG}"
CHECKIN_IMAGE="${ECR_REGISTRY}/${CHECKIN_REPO}:${IMAGE_TAG}"
FRONTEND_IMAGE="${ECR_REGISTRY}/${FRONTEND_REPO}:${IMAGE_TAG}"

sed -i -E "s|^([[:space:]])*image:([[:space:]])*.*$|\1${RESERVATION_IMAGE}|" gitops/reservation-
deployment.yaml
sed -i -E "s|^([[:space:]])*image:([[:space:]])*.*$|\1${CHECKIN_IMAGE}|" gitops/checkin-deployment.yaml
sed -i -E "s|^([[:space:]])*image:([[:space:]])*.*$|\1${FRONTEND_IMAGE}|" gitops/frontend-deployment.yaml

git add gitops/reservation-deployment.yaml gitops/checkin-deployment.yaml gitops/frontend-deployment.
yaml
git commit -m "Update application images to build ${IMAGE_TAG}"
git push origin HEAD:${GIT_BRANCH}
```

Use Jenkins credentials for the GitHub push. Never put a PAT directly into the Jenkinsfile. The implemented pipeline disables shell tracing around the push URL to reduce token exposure in console output.

18. Argo CD -> EKS Deployment Verification

18.1 Check application state

```
argocd app get flight-reservation
kubectl get deployments -n flight-reservation
kubectl get pods -n flight-reservation
kubectl get svc -n flight-reservation
```

18.2 Verify deployed image

```
kubectl get deployment flight-frontend -n flight-reservation -o jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

18.3 Watch rollouts

```
kubectl rollout status deployment/flight-reservation-app -n flight-reservation
kubectl rollout status deployment/flight-checkin-app -n flight-reservation
kubectl rollout status deployment/flight-frontend -n flight-reservation
```

18.4 Get the frontend LoadBalancer hostname

```
kubectl get svc flight-frontend -n flight-reservation
```

Copy the EXTERNAL-IP value for flight-frontend. If it is a DNS name, use it as the browser URL and as FRONTEND_URL. Do not hard-code an old EC2 public IP.

18.5 Argo CD UI access through a controlled tunnel

```
VM2: kubectl port-forward -n argocd svc/argocd-server 8080:443 --address 127.0.0.1
Windows PowerShell: ssh -i "C:\path\to\key.pem" -N -L 8080:127.0.0.1:8080 root@<VM2_PUBLIC_IP>
Browser: https://localhost:8080
```

19. Phase 13 - Prometheus and Grafana

19.1 Install kube-prometheus-stack

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm search repo prometheus-community/kube-prometheus-stack
kubectl create namespace monitoring
helm install monitoring prometheus-community/kube-prometheus-stack --namespace monitoring
```

19.2 Verify monitoring pods and services

```
kubectl get pods -n monitoring
kubectl get svc -n monitoring
helm list -A
```

Expected components include Prometheus, Grafana, Alertmanager, node-exporter, kube-state-metrics and the Prometheus operator.

19.3 Grafana access

```
kubectl get svc monitoring-grafana -n monitoring
kubectl port-forward svc/monitoring-grafana -n monitoring 3000:80 --address=127.0.0.1
```

Windows PowerShell: `ssh -i "C:\path\to\key.pem" -N -L 3000:127.0.0.1:3000 root@<VM2_PUBLIC_IP>`
Browser: `http://localhost:3000`

19.4 Get Grafana credentials

```
kubectl get secret monitoring-grafana -n monitoring -o jsonpath='{.data.admin-user}' | base64 -d; echo
kubectl get secret monitoring-grafana -n monitoring -o jsonpath='{.data.admin-password}' | base64 -d;
echo
```

19.5 Prometheus UI

```
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090 --address=127.0.0.1
```

Windows PowerShell: `ssh -i "C:\path\to\key.pem" -N -L 9090:127.0.0.1:9090 root@<VM2_PUBLIC_IP>`
Browser: `http://localhost:9090`

If port 9090 is already in use, check it with `ss -ltnp | grep :9090`. An existing kubectl port-forward can be reused, or choose another local port such as 9091.

19.6 Useful PromQL queries

up

```
kube_pod_info{namespace="flight-reservation"}
```

```
kube_pod_status_ready{namespace="flight-reservation",condition="true"}
```

```
sum by (pod) (rate(container_cpu_usage_seconds_total{namespace="flight-reservation",container!="",
container!="POD"}[5m]))
```

```
sum by (pod) (container_memory_working_set_bytes{namespace="flight-reservation",container!="",
container!="POD"})
```

```
sum by (pod) (kube_pod_container_status_restarts_total{namespace="flight-reservation"})
```

```
kube_deployment_status_replicas_available{namespace="flight-reservation"}
```

20. Phase 15 - CloudWatch and SNS

Terraform configures AWS-side monitoring for EC2 and RDS. The intended flow is CloudWatch Alarm -> SNS Topic -> Email subscription.

20.1 Verify CloudWatch alarms

```
aws cloudwatch describe-alarms --region ap-south-1 --query 'MetricAlarms[].{Name:AlarmName,State:StateValue}' --output table
```

20.2 Verify SNS

```
aws sns list-subscriptions --region ap-south-1 --output table
```

An SNS email subscription normally requires confirmation from the target inbox before expecting alert emails.

20.3 Example alert path

```
RDS CPU > threshold
-> CloudWatch Alarm enters ALARM
-> SNS notification
-> Email
```

21. Phase 16 - End-to-End CI/CD Test

```
Developer
-> git push
-> GitHub main
-> Jenkins
    -> Checkout
    -> Backend build
    -> Frontend build
    -> SonarQube
    -> Docker build
    -> ECR push
    -> Update GitOps
    -> Git push
-> Argo CD
    -> EKS rollout
    -> new pods
    -> ECR images
```

21.1 Deployment checks

```
kubectll get pods -n flight-reservation
kubectll rollout status deployment/flight-frontend -n flight-reservation
kubectll rollout status deployment/flight-reservation-app -n flight-reservation
kubectll rollout status deployment/flight-checkin-app -n flight-reservation
kubectll get svc -n flight-reservation
```

21.2 Application logs

```
kubectll logs deployment/flight-reservation-app -n flight-reservation --tail=100
kubectll logs deployment/flight-checkin-app -n flight-reservation --tail=100
```

22. Verify That Website Data Reaches RDS

Use VM1, because it can reach the private RDS endpoint. First verify that the database schema exists, then perform a real application action and query the resulting rows.

22.1 Reservation database tables

```
mysql -h <RDS_ENDPOINT> -u flightadmin -p flightdb -e "SHOW TABLES;"
```

The implemented Reservation database showed: booking, flight, user and user_roles.

22.2 Count rows before a test

```
mysql -h <RDS_ENDPOINT> -u flightadmin -p flightdb -e "SELECT COUNT(*) AS bookings FROM booking;"
mysql -h <RDS_ENDPOINT> -u flightadmin -p flightdb -e "SELECT COUNT(*) AS flights FROM flight;"
mysql -h <RDS_ENDPOINT> -u flightadmin -p flightdb -e "SELECT COUNT(*) AS users FROM user;"
```

22.3 Perform website actions

Register a test user and create a test booking from the frontend LoadBalancer URL. Then run the same queries again. For a direct inspection of the latest rows, use:

```
mysql -h <RDS_ENDPOINT> -u flightadmin -p flightdb -e "SELECT * FROM booking ORDER BY id DESC LIMIT 10;"
mysql -h <RDS_ENDPOINT> -u flightadmin -p flightdb -e "SELECT * FROM user ORDER BY id DESC LIMIT 10;"
```

If row counts do not change, check Reservation/Check-In pod logs at the exact time of the operation and verify that the pods received the expected datasource environment variables. A Running pod only proves that the container started; it does not prove every INSERT succeeded.

23. Troubleshooting Quick Reference

Jenkins checkout Permission denied

Workspace contains files owned by root. Remove the Jenkins job workspace and recreate it with Jenkins ownership. Example: `rm -rf /var/lib/jenkins/workspace/flight-reservation-app-AWS; mkdir -p ...; chown -R jenkins:jenkins ...`

SonarQube withSonarQubeEnv unavailable

Install the Jenkins SonarQube Scanner integration and configure a server named Sonarqube.

SonarQube URL says http://http://HOST:9000

Fix Jenkins System -> SonarQube servers -> Server URL to exactly `http://HOST:9000`.

Reservation pod CreateContainerConfigError

Describe the pod. Typical cause: missing flight-reservation-secrets Secret referenced by the Deployment.

Old placeholder RDS URL in ConfigMap

Replace in `gitops/configmap.yaml` with the real private RDS endpoint and push the Git commit.

Frontend placeholder URL

Do not guess the new LoadBalancer hostname. After the Service is created, run `kubectl get svc flight-frontend -n flight-reservation` and copy EXTERNAL-IP.

H2 test error uses RDS URL

Keep H2 properties under `src/test/resources/application.properties` and remove/rename a stray `project-root/application.properties` that overrides test configuration.

Maven duplicate H2 warning

Remove the duplicate H2 dependency from `pom.xml`. Keep one test-scope declaration.

Prometheus/Grafana UI cannot be opened

Use `kubectl port-forward` and an SSH local tunnel. Do not expose another public LoadBalancer just for admin UI unless that is an intentional network decision.

Port 9090 already in use

Check `ss -ltnp | grep :9090` and reuse the existing port-forward or choose a different local port.

Jenkins image update did not reach EKS

`git pull origin main`; verify GitOps image lines; check `argocd app get flight-reservation`; inspect the frontend deployment image.

24. Final Validation Checklist

Area	Expected result
Terraform	terraform validate succeeds
VPC	flight-reservation-dev-vpc exists
EC2	Two C7i-flex.large VMs running as intended
Jenkins	systemctl status jenkins shows running
Docker	docker ps / docker build works on VM1
SonarQube	Jenkins can reach configured SonarQube server
EKS	kubectrl get nodes shows two Ready workers
RDS	flight-reservation-dev-rds available and private
RDS DBs	flightdb + checkin_db exist
S3	Bucket exists with intended access controls
ECR	Three application repositories exist
Argo CD	flight-reservation Synced and Healthy
Application	Frontend + Reservation + Check-In pods Running
Frontend	LoadBalancer has an EXTERNAL-IP/DNS name
Prometheus	Monitoring pods Running and queries return data
Grafana	UI opens and Prometheus datasource test succeeds
CloudWatch	Expected alarms are configured
SNS	Email subscription is confirmed
Database	Website-created data appears in RDS tables

Final mental model: CODE (GitHub) -> BUILD (Jenkins + SonarQube + Docker) -> STORE (ECR / RDS / S3) -> DEPLOY (GitOps + Argo CD + EKS) -> RUN (Frontend + Reservation + Check-In) -> MONITOR (Prometheus + Grafana; CloudWatch + SNS)

25. GitHub Evidence / Screenshots

Recommended evidence for the repository README or docs/screenshots directory:

- Terraform apply / infrastructure output.
- EC2 and EKS verification.
- Jenkins successful pipeline with all stages.
- SonarQube project/analysis.
- ECR repositories/images.
- Argo CD application showing Synced/Healthy.
- `kubectl get pods -n flight-reservation` showing three Running pods.
- Frontend LoadBalancer Service showing EXTERNAL-IP.
- Prometheus Targets page showing UP targets.
- Prometheus query/graph for up or kube_pod_info.
- Grafana Explore with Prometheus datasource.
- Grafana dashboard showing CPU, memory, pod status and restarts.
- RDS table/query proving application data was stored.
- CloudWatch alarms and SNS subscription confirmation.

Do not include passwords, GitHub PATs, SSH private keys, database credentials, or other secrets in screenshots or README files.

26. Cleanup / Destroy When the Project Is No Longer Needed

Use cleanup only when you intentionally want to decommission the environment. Terraform destroy removes Terraform-managed resources; manually installed Helm/Argo/Kubernetes resources must be removed separately.

26.1 Remove application and monitoring resources first

```
VM2:
argocd app delete flight-reservation --cascade
kubectl delete namespace flight-reservation
helm uninstall monitoring -n monitoring
kubectl delete namespace monitoring
# Remove argocd namespace only if you intentionally want to remove Argo CD itself:
kubectl delete namespace argocd
```

26.2 Review Terraform state before destroy

```
cd ~/flight-reservation-app-AWS/Terraform
terraform state list
terraform plan -destroy -out=destroy.tfplan
terraform show -no-color destroy.tfplan | tail -80
```

Review the destroy plan carefully before executing it. If the environment contains data you need, back it up first. ECR images, S3 contents, and RDS deletion behavior can depend on Terraform settings and existing data.

26.3 Apply the reviewed destroy plan

```
terraform apply "destroy.tfplan"
```

26.4 Verify

```
aws ec2 describe-instances --region ap-south-1 --output table
aws eks list-clusters --region ap-south-1
aws rds describe-db-instances --region ap-south-1 --output table
aws ecr describe-repositories --region ap-south-1 --output table
aws s3api list-buckets --output table
```

Never delete the RDS instance or application data just to fix a deployment problem. Use normal GitOps/Kubernetes troubleshooting first.

27. One-Page Command Sheet

```
=== TERRAFORM ===
cd ~/flight-reservation-app-AWS/Terraform
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=flight-reservation-dev.tfplan
terraform show -no-color flight-reservation-dev.tfplan | tail -40
terraform apply "flight-reservation-dev.tfplan"
terraform output

=== AWS IDENTITY ===
aws sts get-caller-identity

=== EKS / VM2 ===
aws eks update-kubeconfig --name flight-reservation-dev-eks --region ap-south-1
kubectl get nodes -o wide

=== ARGO CD ===
kubectl get applications -n argocd
argocd app get flight-reservation
argocd app sync flight-reservation

=== APP ===
kubectl get deployments -n flight-reservation
kubectl get pods -n flight-reservation
kubectl get svc -n flight-reservation

=== ECR ===
aws ecr describe-repositories --region ap-south-1 --output table

=== RDS ===
aws rds describe-db-instances --region ap-south-1 --db-instance-identifier flight-reservation-dev-rds --
output table

=== S3 ===
aws s3api list-buckets --output table

=== PROMETHEUS / GRAFANA ===
kubectl get pods -n monitoring
kubectl get svc -n monitoring
helm list -A

=== CLOUDWATCH / SNS ===
aws cloudwatch describe-alarms --region ap-south-1 --output table
aws sns list-subscriptions --region ap-south-1 --output table

=== LOGS ===
kubectl logs deployment/flight-reservation-app -n flight-reservation --tail=100
kubectl logs deployment/flight-checkin-app -n flight-reservation --tail=100
```

Source basis: Flight_Reservation_AWS_DevOps_Step_by_Step_Runbook.pdf plus the actual AWS implementation/troubleshooting steps completed during this project. Commands above intentionally use placeholders for secrets and environment-specific values.